

CMPEN/EE 454, Project 1, Fall 2026

1 Motivation

The goal of this project is to implement the forward pass of a convolutional neural net (CNN) for object recognition in Matlab. One of the eye-opening developments in computer vision in the last decade is that a cascade or chain of very simple image processing operations can outperform more complicated state-of-the-art object recognition algorithms. Of course, the specific image processing operations used have to be well-chosen! The real magic in a CNN is learning the underlying convolution filters, using a process called backpropagation (we'll call this the backward pass). Much of the heavy computational and big-data requirements for training CNNs (so-called deep learning) are concentrated on this learning process, which is outside the scope of this course and project. However, once learned, applying a CNN in the forward direction to map an input image through a set of intermediate image processing transformations and finally to a set of object class probabilities involves little more than convolution with linear filters, image size reduction, and thresholding. This chain of operations is what we are going to implement in this project.

Specific project goals include:

- Gaining experience in Matlab programming for implementing vision algorithms
- Use of multi-dimensional arrays in Matlab
- Applying linear filters to an image with convolution
- Other image processing techniques: normalization, subsampling, thresholding
- Quantitative performance evaluation of a vision algorithm
- Recording a narrative video to advertise/demonstrate your work

2 The Basic Operations

A CNN operates in a set of stages, or layers, each of which produces an intermediate result. Input to each stage is an $N_1 \times M_1 \times D_1$ array, and output is an $N_2 \times M_2 \times D_2$ array. Input to the first stage is a color image (e.g. array of size 32x32x3); output from the final stage is a vector of probabilities, one per object class (e.g. array of size 1x1x10 if we have 10 different object classes), with higher probabilities meaning it is likely that the picture contains that kind of object. We are going to use computer vision nomenclature and think of an $N \times M \times D$ array as a multi-channel image having N rows, M columns, and D channels. *Channels* is just a word to describe the number of values associated with each 2D spatial pixel location in an image, so, for example, a greyscale image has 1 channel and a color image has 3 channels (red, green, blue). An image with D channels might be used to represent a multispectral image from a sensor that measures D wavelength ranges. In our case, D channels just means that you need D numbers to fully describe the value stored at each spatial (row, column) pixel location.

Different CNNs in the literature are made up of various numbers of layers, operating on images of various sizes, using various combinations/orderings of a basic library of computations. Our simple CNN is going to be built from the following computational building blocks.

Image Normalization: Input is a color image array of size $N \times M \times 3$ and output is an array of the same size. Red, green and blue input channels are unsigned 8 bit integers. We want to first convert them to double float numbers between 0 and 1

$$\text{Tmp}(i, j, k) = \text{In}(i, j, k) / 255.0 \ .$$

Then, using mean and standard deviation values pre-computed over the whole dataset, we subtract the mean and divide by standard deviation over each of the three color channels. For CIFAR-10, the hardcoded means are $(\mu_R, \mu_G, \mu_B) = (0.4914, 0.4822, 0.4465)$ and the standard deviations are $(\sigma_R, \sigma_G, \sigma_B) = (0.2470, 0.2435, 0.2616)$. Thus

$$\text{Out}(i, j, 1) = (\text{Tmp}(i, j, 1)) - \mu_R / \sigma_R$$

and similarly for the G and B channels.

For example, a pixel with original values $(R, G, B) = (128, 64, 200)$ is first converted to

$$(R, G, B) = \left(\frac{128}{255}, \frac{64}{255}, \frac{200}{255} \right) \approx (0.5020, 0.2510, 0.7843)$$

from which the normalized pixel value is then computed as

$$\left(\frac{0.5020 - 0.4914}{0.2470}, \frac{0.2510 - 0.4822}{0.2435}, \frac{0.7843 - 0.4465}{0.2616} \right) \approx (0.043, -0.950, 1.290).$$

ReLU: Input is an array of size $N \times M \times D$ and output is an array of the same size. Each value in the output array is defined as

$$\text{Out}(i, j, k) = \max(\text{In}(i, j, k), 0) \ .$$

ReLU stands for Rectified Linear Unit, but despite the fancy name it is just a thresholding operation where any negative numbers in the input become 0 in the output.

Maxpool: Input is an array of size $2N \times 2M \times D$ and output is an array of the size $N \times M \times D$. If we think of the first two dimensions of an array as being rows and columns, then maxpool is reducing the spatial size of the array by producing an output having only half the number of rows and columns (so total number of elements in the array will be reduced by 4). Note that the number of channels, D, in the output array is the same as number of channels in the input array, so only the spatial dimensions are being reduced. Maxpool operates on each channel separately, and assigns output channel values by taking the maximum value in each nonoverlapping 2x2 block of the input channel. More precisely,

$$\text{Out}(i, j, k) = \max(\{ \text{In}(r, c, k) \mid (2i-1) < r < 2i \text{ and } (2j-1) < c < 2j \}) \ .$$

Figure 1 shows an example of reducing one channel of a 4x4 input image to get one channel of a 2x2 output image. You may assume the input image has an even number of rows and columns.

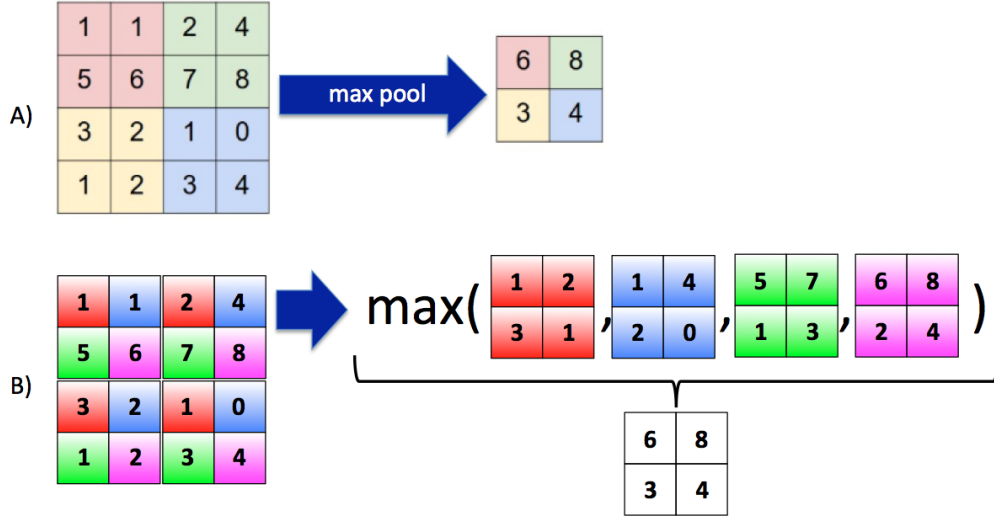


Figure 1: Example of using maxpool to reduce a 4x4 image to a 2x2 image. A) Each nonoverlapping 2x2 block of the image is replaced by the max over the 4 values in that block. B) In an alternative way of doing the computation, perhaps more suitable for Matlab implementation, the 4x4 image is converted into four 2x2 images by taking every other pixel value, starting at locations (1,1) , (1,2) , (2,1) and (2,2). A final 2x2 image is produced by taking the max over corresponding pixel locations in those 4 reduced images. Note: although this is conceptually what you want to do, in practice you can't apply max to 4 arrays at the same time, only 2 at a time.

Convolution: Input is an array of size $N \times M \times D_1$ and output is an array of size $N \times M \times D_2$. Note that the spatial size of the image stays the same (number of rows and columns are the same), but the number of channels in the output image can differ from the input. The job of a convolution layer is to apply a set of predefined linear filters and bias values to the input image to compute each channel of the output image. This is the main computational process in a CNN, and the most difficult to implement.

To fully specify the linear transformation from input to output, we will need to know a set of D_2 scalar bias values and a set of D_2 filters of size $R \times C \times D_1$, where $R \times C$ is the spatial size of a filter, and typically is small compared to the $N \times M$ spatial size of the image. We will call the set of filters a *filter bank*, and represent it in Matlab as a 4-dimensional array of size $R \times C \times D_1 \times D_2$. That is, if F is our filter bank array, then $F(:, :, :, l)$ is the l th filter, $1 \leq l \leq D_2$, and that filter is a 3-dimensional array of size $R \times C \times D_1$. This l th filter is applied to the entire $N \times M \times D_1$ input image to compute a single $N \times M$ channel of the output image, to which the l th bias value is then added. Because there are D_2 filters (and bias values), this means the output image will have D_2 channels, and will therefore be of size $N \times M \times D_2$. See Figure 2.

Let F_l be the l th filter and b_l be the l th scalar bias value. We compute the l th channel of the output image using summations of 2D convolutions, defined as follows:

$$\text{Out}(:, :, l) = \sum_{k=1}^{D_1} F_l(:, :, k) * \text{In}(:, :, k) + b_l .$$

In words, each channel $1 \leq k \leq D_1$ of the input image is convolved with the corresponding channel

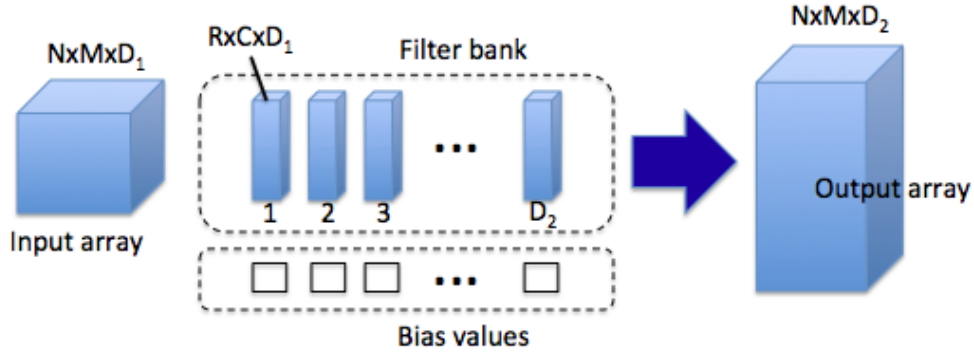


Figure 2: A convolution layer transforms an $N \times M \times D_1$ input array into an $N \times M \times D_2$ output array by convolving with a filter bank of D_2 linear filters and adding bias values. See text for details.

of the filter, those D_1 result images are summed up, and then the scalar bias value is added. This process is carried out D_2 times, for $1 \leq l \leq D_2$, to fill in all D_2 channels of the output image. When we say “convolution” here, we really mean convolution, for example using `imfilter` with the ‘conv’ option; the filters you will be given will not work correctly if you use cross-correlation. In addition, you want the size of each convolved image to remain $N \times M$, and you also want to use zero-padding.

Fully Connected: Input is an array of size $N \times M \times D_1$ and output is an array of size $1 \times 1 \times D_2$. Similar to the convolution layer, the fully connected layer applies a filter bank of D_2 linear filters and D_2 scalar bias values to compute output values. However, unlike convolution layers, each filter is the same size as the input image, and is applied in a way that computes a single, scalar valued output. Because there are D_2 filters and bias values, the output image will contain D_2 scalar values, which we will represent as a $1 \times 1 \times D_2$ array. See Figure 2. The term “fully connected” comes from the traditional neural network literature – each scalar output value is computed as a linear combination of the full set of $N \times M \times D_1$ input values.

Given an $N \times M \times D_1 \times D_2$ filter bank array and set of D_2 scalar bias values, let F_l be the l th filter and b_l be the l th bias value. We compute the l th channel of the output image as the scalar value:

$$\text{Out}(1, 1, l) = \sum_{i=1}^N \sum_{j=1}^M \sum_{k=1}^{D_1} F_l(i, j, k) \times \text{In}(i, j, k) + b_l .$$

In words, the l th output value is computed by multiplying every value in the input image by a corresponding filter weight, summing all of those terms up, and then adding the bias value. This is repeated for each filter F_l and bias b_l , $1 \leq l \leq D_2$, to compute all D_2 output values. Since each filter has the same number of values in it as the input image, if you were to rearrange each of them into a 1D vector of length NMD_1 you would recognize the equation above as performing a dot product operation followed by adding a scalar bias value. Whether or not you choose to compute it that way is up to you.

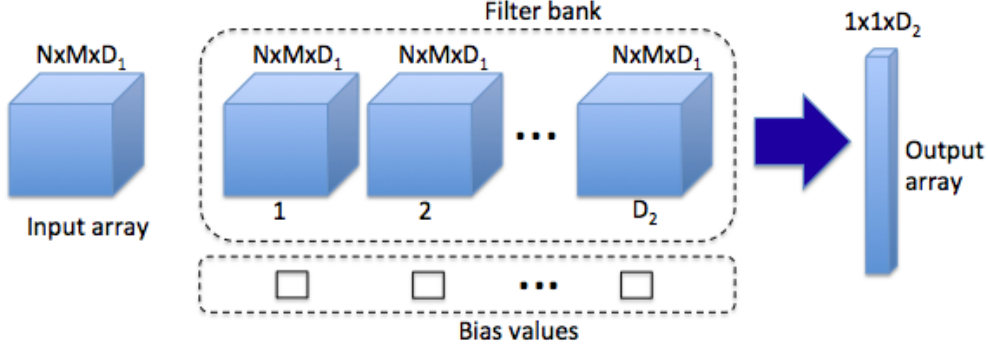


Figure 3: A fully connected layer transforms an $N \times M \times D_1$ input array into a $1 \times 1 \times D_2$ output array (basically a vector of numbers) by applying a filter bank of D_2 linear filters and additive bias values. Applying each filter is similar to performing a dot product operation. See text for details.

Softmax: Input is an array of size $1 \times 1 \times D$ and output is an array of the same size. Each value in the output array is defined as

$$\text{Out}(1, 1, k) = \frac{\exp(\text{In}(1, 1, k) - \alpha)}{\sum_{k=1}^D \exp(\text{In}(1, 1, k) - \alpha)}$$

where

$$\alpha = \max_k \text{In}(1, 1, k)$$

is the maximum across all values in the input vector. Softmax takes a vector of arbitrary real numbers and converts them into numbers that can be viewed as probabilities, that is, all will lie between 0 and 1 and sum up to 1. Usually you will see softmax defined mathematically without the α term. However, you want to use our definition when writing a program, as it is numerically more stable.

3 Putting It All Together

Now that we know the basic building block functions, it is time to put them all together to do object recognition. We will implement an 18-layer CNN that takes a 32×32 color image as input and produces 10 object class probabilities as output. The CNN will apply a particular ordered sequence of the basic operations above, step by step, to ultimately produce the output vector of class probabilities. Each operation is one “layer” of the computational chain, with each operation taking as input the output from the previous layer. The number of layers is called the “depth” of the CNN. Because the layer computations are all fairly simple, you need a lot of layers to accomplish anything interesting, and thus working CNNs tend to be deep (have a lot of computational depth). This is why training them is called deep learning.

Dataset: The images we will be using come from the well-known CIFAR-10 dataset. Despite being a “toy” dataset with thumbnail-sized images, only 10 object classes, and only one object per

image, results on CIFAR are often presented by machine learning practitioners because the dataset is so familiar to everyone in the field. The object classes and some sample images from each class are shown in Figure 4.

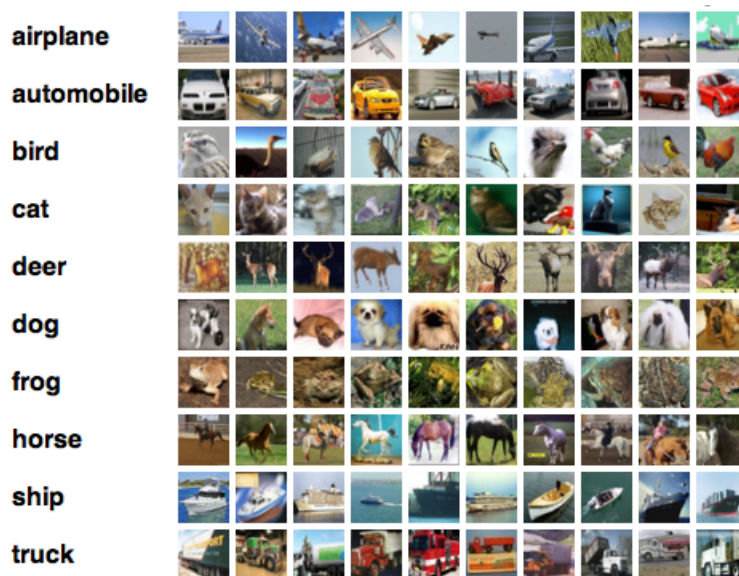


Figure 4: Sample images from each of the 10 object classes in CIFAR-10.

In the project directory, you will find a file called “cifar10testdata.mat” that contains a set of 10000 test images, 1000 from each class, along with their ground truth object classifications. Some sample code that illustrates reading in and dealing with the data format is shown below.

```
%loading this file defines imageset, trueclass, and classlabels
load 'cifar10testdata.mat'

%some sample code to read and display one image from each class
for classindex = 1:10
    %get indices of all images of that class
    inds = find(trueclass==classindex);
    %take first one
    imrgb = imageset(:, :, :, inds(1));
    %display it along with ground truth text label
    figure; imagesc(imrgb); truesize(gcf, [64 64]);
    title(sprintf('\%s', classlabels{classindex}));
end
```

CNN layers and parameters: Table 1 specifies each of the 18 layer computations that our CNN will perform. Sizes of each input and output array are shown, and for the convolution and fully connected layers the size of the filter bank and number of scalar bias values are also shown. Note that the bulk of the computations consist of conv-ReLU pairs, with an occasional maxpool thrown in to progressively reduce the image sizes being processed. Image normalization is done once at the beginning to prepare the input RGB image for processing. One fully connected layer is

done at the end to distill all the results down to 10 numbers that softmax then converts into class probabilities.

layer #	layer type	input size	filterbank size	num biases	output size
1	imnormalize	$32 \times 32 \times 3$			$32 \times 32 \times 3$
2	convolve	$32 \times 32 \times 3$	$3 \times 3 \times 3 \times 10$	10	$32 \times 32 \times 10$
3	relu	$32 \times 32 \times 10$			$32 \times 32 \times 10$
4	convolve	$32 \times 32 \times 10$	$3 \times 3 \times 10 \times 10$	10	$32 \times 32 \times 10$
5	relu	$32 \times 32 \times 10$			$32 \times 32 \times 10$
6	maxpool	$32 \times 32 \times 10$			$16 \times 16 \times 10$
7	convolve	$16 \times 16 \times 10$	$3 \times 3 \times 10 \times 10$	10	$16 \times 16 \times 10$
8	relu	$16 \times 16 \times 10$			$16 \times 16 \times 10$
9	convolve	$16 \times 16 \times 10$	$3 \times 3 \times 10 \times 10$	10	$16 \times 16 \times 10$
10	relu	$16 \times 16 \times 10$			$16 \times 16 \times 10$
11	maxpool	$16 \times 16 \times 10$			$8 \times 8 \times 10$
12	convolve	$8 \times 8 \times 10$	$3 \times 3 \times 10 \times 10$	10	$8 \times 8 \times 10$
13	relu	$8 \times 8 \times 10$			$8 \times 8 \times 10$
14	convolve	$8 \times 8 \times 10$	$3 \times 3 \times 10 \times 10$	10	$8 \times 8 \times 10$
15	relu	$8 \times 8 \times 10$			$8 \times 8 \times 10$
16	maxpool	$8 \times 8 \times 10$			$4 \times 4 \times 10$
17	fullconnect	$4 \times 4 \times 10$	$4 \times 4 \times 10 \times 10$	10	$1 \times 1 \times 10$
18	softmax	$1 \times 1 \times 10$			$1 \times 1 \times 10$

Table 1: An 18-layer convolutional neural net.

The filter bank and bias values needed to completely specify the CNN are given to you in a file called “CNNparameters.mat”. Loading this file defines three variables: layertypes, filterbanks and biasvectors. These are cell arrays (indexed with curly braces). For ease of use, there is an entry for each of the 18 layers of the CNN. However, only the cells for layers that involve using filterbank and bias values (the convolution layers and fully connected layer) have values stored in them, the rest are empty. The following sample code illustrates how to read and access the layertypes, filterbanks and bias vectors.

```
%loading this file defines filterbanks and biasvectors
load 'CNNparameters.mat'

%sample code to verify which layers have filters and biases
for d = 1:length(layertypes)
    fprintf('layer %d is of type %s\n',d,layertypes{d});
    filterbank = filterbanks{d};
    if not(isempty(filterbank))
        fprintf('    filterbank size %d x %d x %d x %d\n', ...
            size(filterbank,1),size(filterbank,2), ...
            size(filterbank,3),size(filterbank,4));
```

```

        biasvec = biasvectors{d};
        fprintf('        number of biases is %d\n',length(biasvec));
    end
end

```

We want you to write external functions (each function in a separate, named .m file) to do the computation for each type of layer.

```

in file apply_imnormalize.m you will define
    function outarray = apply_imnormalize (inarray)
    %inarray is an NxMx3 uint8 image and outarray is NxMx3
in file apply_relu.m you will define
    function outarray = apply_relu(inarray)
    %inarray is NxMxD and outarray is the same size
in file apply_maxpool.m you will define
    function outarray = apply_maxpool(inarray)
    %inarray is 2Nx2MxD and outarray is size NxMxD
in file apply_convolve.m you will define
    function outarray = apply_convolve(inarray, filterbank, biasvals)
    %inarray is NxMxD1, filterbank is RxCxD1xD2,
    %biasvals is a length D2 vector, and outarray is NxMxD2
in file apply_fullconnect.m you will define
    function outarray = apply_fullconnect(inarray, filterbank, biasvals)
    %inarray is NxMxD1, filterbank is NxMxD1xD2,
    %biasvals is a length D2 vector, and outarray is 1x1xD2
in file apply_softmax.m you will define
    function outarray = apply_softmax(inarray)
    %inarray is 1x1xD and outarray is the same size

```

It is important that you use the specific filenames and input/output parameter definitions above because we will be writing scripts to test your functions. Unless otherwise specified, all variables are double float.

Of course you will have other functions and a main routine that reads the input files, calls the CNN layer functions in the right order, and analyzes/reports the classification results. You can use your own design choices for implementing those.

Debugging: Finally, to help you develop your code correctly, the file “debuggingTest.mat” contains a sample test image (imrgb) and expected output array produced by each layer of the CNN (layerResults) when that image is used as input. The following sample code illustrates how to read and access this data. I highly recommend verifying that your implementation of each of the algorithmic building blocks described above works as it is supposed to by comparing its output to these known correct results. For example, if you have just finished implementing a maxpool routine and want to test it, make sure that when it is run on the image array in layerResults{5}, it produces the output array in layerResults{6}. When all types of layers have been implemented, you should

be able to start with `imrgb` and run the whole CNN from end to end to get the probabilities in `layerResults{18}`.

```
%loading this file defines imrgb and layerResults
load 'debuggingTest.mat'

%sample code to show image and access expected results
figure; imagesc(imrgb); truesize(gcf,[64 64]);
for d = 1:length(layerResults)
    result = layerResults{d};
    fprintf('layer %d output is size %d x %d x %d\n',...
        d,size(result,1),size(result,2), size(result,3));
end
%find most probable class
classprobvec = squeeze(layerResults{end});
[maxprob,maxclass] = max(classprobvec);
%note, classlabels is defined in 'cifar10testdata.mat'
fprintf('estimated class is %s with probability %.4f\n',...
    classlabels{maxclass},maxprob);
```

4 Quantitative Evaluation

How do you know that your algorithm works correctly? There are two meanings of *correct* we could be interested in. One is whether your code is implemented correctly, which you can check by using the information in `'debuggingTest.mat'` to verify that each layer of your CNN is producing the expected results. However, in real life (industry coding; grad school research) it is taken as a given that you are competent enough to implement a program that does what the specifications say it should do. The question then is how well does the method you are implementing perform? This is where the topic of performance evaluation comes in.

There are many ways to evaluate a classifier. What we perhaps most care about in a classifier is overall accuracy, that is, what percentage of test examples does it choose the correct class for. Other questions we might be interested in are whether the approach finds some classes to be easier to classify than others, and whether there are pairs of classes that are frequently confused for each other. All of these questions can be answered by compiling a *confusion matrix* or *contingency table* on a representative test set.

The obvious way for having our CNN classifier choose what object class is in the image is to take the class associated with the highest probability in the output class probability vector (using some heuristic to break ties if necessary). Using the 10000 test image examples in `'cifar10testdata.mat'`, we'd like you to classify each one and accumulate a 10×10 table A_{ij} where each table entry contains the number of times that an example with a ground truth class of i was classified as object class j by the CNN. Values along the diagonal of this table represent examples that were classified correctly (e.g. the image had true class 'airplane' and was classified as 'airplane'). All examples

off the diagonal are errors. The *classification accuracy* or *classification rate* of the CNN can then be computed as

$$\text{Accuracy} = \frac{\sum_i A_{ii}}{\sum_i \sum_j A_{ij}} .$$

Further examination of the confusion matrix can highlight classes that seems easy to classify correctly (rows where the diagonal is a large percentage of the row sum), and pairs of classes that are frequently confused for each other (large off-diagonal values).

5 What Toolboxes and Libraries Can I Use?

Our intent is that you will implement the various computational modules described earlier using “general” array processing and indexing functions in Matlab. One notable exception is that you should use “imfilter” from the image processing toolbox to do convolution, rather than having to implement convolution directly using quadruply nested loops with complicated indexing, which would be a mess. Definitely DO NOT use anything from the computer vision toolbox, neural network toolbox, or deep learning toolbox. Also don’t use any third party libraries or packages specifically created for doing convolutional neural networks (e.g. don’t use matconvnet) and don’t download stuff from GitHub or Matlab Central. Follow our definitions when writing your routines. We will be suspicious of a CNN routine that is more general than it needs to be, as that suggests you copied it from somewhere else.

Acceptable usage of AI: We are aware that doing a simple Google search these days invokes an LLM chatbot that, even if you don’t want it, sticks an AI answer at the top of the results page. It simply can’t be avoided. Using AI to look up common matlab functions and how to use them, or to explain concepts to you, is acceptable. For example, “what does ReLU stand for and what does it do” is acceptable question to ask. We DO NOT WANT you to use AI to solve subproblems. For example, “write me a function that does ReLU” is NOT an acceptable use of AI. This restriction should not come as a surprise, and is analogous to not wanting you to go on coding forums to look up code that someone else has previously written. It is plagiarism even if AI is the author.

6 Grading

Half of your grade will be based on submitting run-able Matlab code, and the other half is based on submitting a short narrative video presenting your team’s understanding of the project, design decisions, results, and experimental observations.

1. Turn in a running version of your main program and all functions in a single zip or tar archive file (e.g. put all code in a single directory, then make a zip file of that directory for submission via Canvas). We can then unzip/untar everything and go from there. In addition to the specific functions outlined earlier that apply each type of CNN layer, also include one or more clearly marked demo routines that can be invoked with no arguments and that load any input needed from the local directory and display intermediate and final results that you think demonstrate that your CNN is

working as intended. We will run your demo routines, and test each of your CNN layer functions for correctness. Include enough comments in your functions so that we have a good idea what each section of code does. The more thought and effort you put into demonstrating/illustrating in your demo routines and narrative video that your code works correctly, the less likely we will feel the need to poke around in your code, but in case we do, make your code and comments readable.

2. Include in the zip file above a short (roughly 5 minutes) narrative video in which you present and discuss at least the following:

a) Summarize *in your own words* what you think the project was about. What were the tasks you performed; what did you expect to achieve?

b) Present an outline of the procedural approach along with a flowchart showing the flow of control and subroutine structure of your Matlab code. Explain any design decisions you had to make. Even though the mathematical specification of each part of this project is fairly strict, there are a lot of different ways you could implement each building block in Matlab, ranging from C-like nested for-loop computations, to cleverly vectorized code. Be sure to document any deviations you made from the above project descriptions (and why), or any additional functionality you added to increase robustness or generality of the approach.

c) Experimental observations. What do you observe about the behavior of your program when you run it? Does it seem to work the way you think it should? Run the different portions of your code and show pictures of intermediate and final results that convince us that the program does what you think it does. Each channel of an intermediate result array in the CNN can be interpreted as a greyscale image. So, for example, the output from layer 2 of the CNN is an array of size $32 \times 32 \times 10$, and this could be displayed as 10 small greyscale images, each of size 32×32 . CNN practitioners often display these intermediate results and interpret them as images showing the results of learned “feature” detectors. The results from the final softmax layer could be displayed as a bar chart. Note that a video presentation is an excellent medium for *visual* displays of your results.

d) Run performance evaluation experiments as described above to compute the confusion matrix and classification rate. Display them and discuss what you have found.

e) Tell us what each team member did to contribute to the project. It is OK if you divide up the labor into different tasks (actually it is expected), and it is OK if not everyone contributes precisely equal amounts of time/effort on each project. However, if two people did all the work because the third team member could not be contacted until the day before the project was due, this is where you get to tell us about it, so the grading can reflect the inequity.

It will actually be fairly challenging to show and tell all of the above within only 5 minutes, but if you do run over time, try not to go too far over. The time limit is there to focus your presentation. There are several ways to generate a video with an audio narration track. One of easiest is to use zoom to record yourself showing slides, programs and/or video results on your computer while you talk about what you are showing, but feel free to use other recording/editing software if you like. It is not necessary for all group members to talk – you can nominate one member to do the narration, or you can take turns talking, it is up to you.

To summarize: You will be submitting a single big zip file that contains all of your working code and a narrated video. We will take it from there!

7 Extra Credit

There are two additional tasks you can optionally choose to do for extra credit.

1) Above, we computed top-1 classification accuracy, based on measuring how often the top scoring object class matched the correct ground truth class. This is a very restrictive test, and often we are interested in relaxing that measure to see how well the classifier would do if it was allowed extra guesses at predicting the correct object class. Implement an evaluation that plots a curve showing classification accuracy with respect to the top- k classes. This would be a plot where the x axis ranges from 1 to 10, and the y axis ranges from 0 to 100 percent. For each integer value k on the x-axis, you would compute and plot as $y(k)$ the percentage of times that the correct object class appeared in the top- k ranked classes, as determined by probability scores sorted from highest to lowest. The plotted value for $k = 1$ is just the usual classification accuracy described above. The plotted value for $k = 2$ is the percentage of times that the correct class for an image is one of the 2 classes that have the highest computed probability scores for that image. And so on. For $k = 10$ the curve would be at 100 percent accuracy, obviously.

2) The `imnormalize` routine that we have described above is kind of a special case hack that normalizes pixel values based on hardcoded R,G,B mean and std values, using a special case function that only gets used once in the network chain. Describe in words and math how you could implement our `imnormalize` routine as a convolution layer. What would the filterbank be (what size; what values) and what would the bias vector be (what size; what values) to enable `imnormalize` to be just another convolution layer?